

Sistema de Alocação de Salas de Aula

Documentação Técnica do Modelo de Otimização

Instituto de Ciências Matemáticas e de Computação (ICMC)

13 de abril de 2026

Sumário

1	Introdução	3
1.1	Objetivos do Sistema	3
2	Arquitetura do Sistema	3
2.1	Bibliotecas Utilizadas	3
2.2	Fluxo de Execução	3
3	Estrutura de Dados	3
3.1	Entrada de Dados	3
3.1.1	Campos das Disciplinas	4
3.1.2	Campos das Salas	4
4	Processamento de Horários	4
4.1	Conversão de Horários	4
4.2	Detecção de Conflitos	4
4.3	Identificação de Aulas Seguidas	5
5	Modelo Matemático	5
5.1	Conjuntos	5
5.2	Parâmetros	5
5.3	Variáveis de Decisão	5
5.3.1	Variáveis Principais	5
5.3.2	Variáveis Auxiliares	6
5.4	Função Objetivo	6
5.5	Restrições	6
5.5.1	Restrição (3.3): Alocação Única	6
5.5.2	Restrição (3.4): Conflitos de Horário	7
5.5.3	Restrição de Contagem de Salas	7
5.5.4	Restrição de Superlotação	7
5.5.5	Restrição (3.13): Relação w_{cs} e x_{as}	7
5.5.6	Restrição (3.14): Relação w_{cs} e $v_{css'}$	7
6	Implementação	7
6.1	Parâmetros de Execução	7
6.2	Configurações do Solver	8

7	Saídas do Sistema	8
7.1	Arquivo: Dados da Solução do Modelo	8
7.2	Arquivo: Distribuição de Cursos	8
8	Tratamento de Casos Especiais	9
8.1	Laboratórios	9
8.2	Salas Fixadas	9
8.3	Salas Proibidas	9
9	Validação e Verificação	9
9.1	Verificação de Factibilidade	9
9.2	Métricas de Qualidade	9
10	Tratamento de Erros	10
10.1	Erros de Permissão	10
10.2	Validação de Dados	10
11	Considerações de Desempenho	10
11.1	Complexidade	10
11.2	Técnicas de Otimização	10
12	Manutenção e Extensões	10
12.1	Adição de Novos Cursos	10
12.2	Adição de Novas Restrições	11
13	Referências e Contato	11
13.1	Tecnologias Utilizadas	11

1 Introdução

Este documento descreve o sistema de alocação automática de salas de aula desenvolvido para o ICMC. O sistema utiliza técnicas de Programação Linear Inteira Mista (PLIM) para otimizar a distribuição de aulas nas salas disponíveis, considerando múltiplas restrições e objetivos.

1.1 Objetivos do Sistema

- Minimizar o espaço vazio nas salas (otimizar ocupação)
- Minimizar trocas de sala para cada disciplina
- Reduzir distâncias entre salas para cursos relacionados
- Evitar superlotação das salas
- Respeitar requisitos de laboratório
- Considerar salas fixadas e restrições específicas

2 Arquitetura do Sistema

2.1 Bibliotecas Utilizadas

```
1 import pandas as pd          # Manipulação de dados
2 import time                  # Cronometragem
3 from mip import *           # Modelagem matemática
4 import openpyxl              # Criação de planilhas
5 from datetime import datetime # Manipulação de horários
```

2.2 Fluxo de Execução

1. **Leitura dos Dados:** Importação das planilhas Excel
2. **Pré-processamento:** Conversão e validação de horários
3. **Construção do Modelo:** Definição de variáveis e restrições
4. **Otimização:** Resolução do modelo matemático
5. **Geração de Saídas:** Criação de relatórios e visualizações

3 Estrutura de Dados

3.1 Entrada de Dados

O sistema lê dados de uma planilha Excel com as seguintes abas:

- **SME, SMA, SCC, SSC, Outros:** Dados das disciplinas por departamento
- **Salas:** Informações sobre capacidade e características das salas

3.1.1 Campos das Disciplinas

Campo	Descrição
Disciplina (código)	Código identificador da disciplina
Disciplina (nome completo)	Nome completo da disciplina
Curso(s)	Cursos para os quais a disciplina é oferecida
Vagas por disciplina	Número de alunos inscritos
Horário 1, 2, 3, 4	Horários das aulas (formato: Dia - HH:MM/HH:MM)
Sala	Sala fixada (se houver)
Utilizará laboratório?	Indicação se precisa de laboratório
Proibir Horário	Salas proibidas para determinado horário

Tabela 1: Campos da planilha de disciplinas

3.1.2 Campos das Salas

Campo	Descrição
Sala	Identificador da sala
Lugares	Capacidade da sala
Laboratório	Indica se é sala de laboratório (Sim/Não)
Preferencialmente Vazia	Peso para preferir sala vazia
Distâncias	Matriz de distâncias entre salas

Tabela 2: Campos da planilha de salas

4 Processamento de Horários

4.1 Conversão de Horários

Os horários são convertidos de formato textual para decimal:

$$h_{decimal} = h_{horas} + \frac{h_{minutos}}{60} \quad (1)$$

Exemplo: $14:20 = 14 + 20/60 = 14.33$

4.2 Detecção de Conflitos

Duas aulas a e a' têm conflito se:

$$\text{dia}_a = \text{dia}_{a'} \wedge (\text{início}_a < \text{fim}_{a'}) \wedge (\text{início}_{a'} < \text{fim}_a) \quad (2)$$

4.3 Identificação de Aulas Seguidas

Aulas são consideradas seguidas se:

$$\text{fim}_a + 2.5 \geq \text{início}_{a'} \quad (3)$$

Onde 2.5 horas representa o intervalo máximo entre aulas consecutivas.

5 Modelo Matemático

5.1 Conjuntos

- T : Conjunto de turmas/disciplinas
- S : Conjunto de salas
- A : Conjunto de aulas
- C : Conjunto de cursos
- A_t : Aulas da turma t
- A_s : Pares de aulas seguidas
- A_c : Aulas do curso c

5.2 Parâmetros

- cap_s : Capacidade da sala s
- tam_t : Tamanho (inscritos) da turma t
- σ_s : Indica se sala s é laboratório
- τ_t : Indica se turma t precisa de laboratório
- η_{as} : Indica se aula a cabe na sala s
- $\theta_{aa'}$: Indica conflito entre aulas a e a'
- uso_{as} : Percentual de espaço vazio se a for alocada em s
- $\text{dis}_{ss'}$: Distância entre salas s e s'
- Y_{tc} : Indica se turma t é ministrada para curso c
- α : Fator de superlotação (ex: $0.85 = 85\%$)

5.3 Variáveis de Decisão

5.3.1 Variáveis Principais

$$x_{as} = \begin{cases} 1, & \text{se aula } a \text{ é alocada na sala } s \\ 0, & \text{caso contrário} \end{cases} \quad (4)$$

$$y_t \in \mathbb{Z}^+ \quad (\text{número de salas/trocas da turma } t) \quad (5)$$

5.3.2 Variáveis Auxiliares

$$c_{st} = \begin{cases} 1, & \text{se sala } s \text{ é usada pela turma } t \\ 0, & \text{caso contrário} \end{cases} \quad (6)$$

$$w_{cs} = \begin{cases} 1, & \text{se curso } c \text{ tem aula na sala } s \\ 0, & \text{caso contrário} \end{cases} \quad (7)$$

$$v_{css'} = \begin{cases} 1, & \text{se curso } c \text{ tem aulas nas salas } s \text{ e } s' \\ 0, & \text{caso contrário} \end{cases} \quad (8)$$

$$z_{as} = \begin{cases} 1, & \text{se aula } a \text{ superlota sala } s \\ 0, & \text{caso contrário} \end{cases} \quad (9)$$

5.4 Função Objetivo

$$\begin{aligned} \min \quad & w_x \sum_{a \in A} \sum_{s \in S} \text{uso}_{as} \cdot x_{as} + w_y \sum_{t \in T} \text{seg}_t \cdot y_t \\ & + w_v \sum_{c \in C} \sum_{s \in S} \sum_{s' \in S, s \neq s'} \text{dis}_{ss'} \cdot v_{css'} \\ & + w_z \sum_{a \in A} \sum_{s \in S} z_{as} + w_p \sum_{a \in A} \sum_{s \in S} \text{pref}_s \cdot x_{as} \end{aligned} \quad (10)$$

Onde:

- w_x : peso para otimização de espaço vazio
- w_y : peso para minimização de trocas/salas
- w_v : peso para minimização de distâncias
- w_z : peso para penalização de superlotação
- w_p : peso para preferências de salas vazias
- seg_t : fator multiplicador (2 se tem aulas seguidas, 1 caso contrário)

5.5 Restrições

5.5.1 Restrição (3.3): Alocação Única

Cada aula deve ser alocada em exatamente uma sala compatível:

$$\sum_{s \in S} x_{as} = \begin{cases} 1, & \text{se aula } a \text{ tem horário definido e inscritos} \\ 0, & \text{caso contrário} \end{cases} \quad \forall a \in A \quad (11)$$

$$x_{as} \leq \eta_{as} \quad \forall a \in A, \forall s \in S \quad (12)$$

5.5.2 Restrição (3.4): Conflitos de Horário

Aulas com conflito não podem ocupar a mesma sala:

$$x_{as} + x_{a's} \leq 1 \quad \forall a, a' \in A : \theta_{aa'} = 1, \forall s \in S \quad (13)$$

Restrições especiais para laboratórios conjuntos:

Para salas 6-303/6-304:

$$x_{a,6-303/6-304} + x_{a',6-303} \leq 1 \quad (14)$$

$$x_{a,6-303/6-304} + x_{a',6-304} \leq 1 \quad (15)$$

5.5.3 Restrição de Contagem de Salas

$$\sum_{a \in A_t} x_{as} \leq |A_t| \cdot c_{st} \quad \forall s \in S, \forall t \in T \quad (16)$$

$$y_t \geq \sum_{s \in S} c_{st} \quad \forall t \in T \quad (17)$$

5.5.4 Restrição de Superlotação

$$\text{tam}_t \cdot x_{as} \leq \alpha \cdot \text{cap}_s + z_{as} \cdot \text{cap}_s \quad \forall a \in A, \forall s \in S \quad (18)$$

5.5.5 Restrição (3.13): Relação w_{cs} e x_{as}

$$x_{as} \leq w_{cs} \quad \forall c \in C, \forall s \in S, \forall a \in A_c \quad (19)$$

5.5.6 Restrição (3.14): Relação w_{cs} e $v_{css'}$

$$2 \cdot v_{css'} \leq w_{cs} + w_{cs'} \quad \forall c \in C, \forall s, s' \in S : s \neq s' \quad (20)$$

$$v_{css'} \geq w_{cs} + w_{cs'} - 1 \quad \forall c \in C, \forall s, s' \in S : s \neq s' \quad (21)$$

6 Implementação

6.1 Parâmetros de Execução

O sistema recebe os seguintes argumentos via linha de comando:

```
python modelo.py <arquivo> <peso_x> <peso_y> <peso_v> <peso_z> <alpha> <peso_pref>
```

- **arquivo:** Caminho para a planilha de entrada
- **peso_x:** Peso para otimização de espaço vazio
- **peso_y:** Peso para minimização de trocas
- **peso_v:** Peso para minimização de distâncias
- **peso_z:** Peso para penalização de superlotação
- **alpha:** Fator de superlotação (0 a 1)
- **peso_pref:** Peso para preferência de salas vazias

6.2 Configurações do Solver

```
1 model.opt_tol = 0.1          # Tolerancia de otimalidade
2 model.verbose = 0           # Desabilita saidas
3 model.tol = 1e-6            # Tolerancia numerica
4 model.integer_tol = 1e-6    # Tolerancia para inteiros
5 model.optimize(max_seconds=7200) # Limite de 2 horas
```

7 Saídas do Sistema

7.1 Arquivo: Dados da Solução do Modelo

Planilha Excel contendo a alocação completa:

- Código da disciplina
- Nome completo
- Cursos atendidos
- Horário da aula
- Sala alocada
- Número de inscritos
- Docente responsável
- NUSP do docente
- Ano dos dados
- Observações (campo livre)

7.2 Arquivo: Distribuição de Cursos

Planilha com visualização de barras mostrando:

- Número de aulas por sala para cada curso
- Barras de progresso coloridas
- Total de aulas por sala
- Ocupação total do sistema
- Tempo de execução

8 Tratamento de Casos Especiais

8.1 Laboratórios

- Identificação automática de disciplinas que requerem laboratório
- Especificação de quais aulas devem ser em laboratório
- Tratamento de salas conjuntas (6-303/6-304, 6-305/6-306)
- Restrições de conflito para salas compartilhadas

8.2 Salas Fixadas

- Leitura de salas pré-definidas (ex: LEM)
- Restrição automática de η_{as} para garantir alocação fixa
- Suporte a múltiplas salas fixadas por disciplina

8.3 Salas Proibidas

- Leitura de restrições específicas por horário
- Ajuste de η_{as} para bloquear salas incompatíveis
- Suporte a lista de salas proibidas

9 Validação e Verificação

9.1 Verificação de Factibilidade

O sistema verifica automaticamente:

```
1 for t in disciplinas:
2     if t not in alocadas:
3         print(f"Disciplina {t} nao foi alocada.")
```

9.2 Métricas de Qualidade

- **Gap de otimalidade:** Distância da solução obtida para o ótimo
- **Ocupação total:** Soma dos espaços vazios
- **Tempo de execução:** Tempo em segundos
- **Status:** OPTIMAL, FEASIBLE, INFEASIBLE, etc.

10 Tratamento de Erros

10.1 Erros de Permissão

```
1 except PermissionError as e:
2     if e.errno == 13:
3         print(f"Arquivo aberto. Feche e tente novamente.")
4         sys.exit(2)
```

10.2 Validação de Dados

- Verificação de células vazias
- Conversão segura de horários
- Tratamento de valores NaN
- Validação de cursos e salas

11 Considerações de Desempenho

11.1 Complexidade

O problema é NP-difícil. A complexidade depende de:

- Número de aulas: $|A|$
- Número de salas: $|S|$
- Número de variáveis: $O(|A| \times |S|)$
- Número de restrições: $O(|A|^2 \times |S|)$

11.2 Técnicas de Otimização

- Pré-processamento de incompatibilidades (η_{as})
- Identificação prévia de conflitos ($\theta_{aa'}$)
- Uso de variáveis auxiliares para linearização
- Definição de tolerâncias apropriadas

12 Manutenção e Extensões

12.1 Adição de Novos Cursos

Adicionar na lista curriculos:

```
1 curriculos = [ 'BMAcc', 'BMA', 'LMA', 'MAT-NG',
2               'BECD', 'BCC', 'BSI', 'BCDados', 'NOVO' ]
```

12.2 Adição de Novas Restrições

```
1 # Exemplo: Restricao de preferencia de docente
2 for t in T:
3     if df.loc[t, 'Preferencia'] == 'Bloco 6':
4         for a in A_t[t]:
5             for s in salas_bloco_6:
6                 model += x_as[a,s] >= condicao
```

13 Referências e Contato

Este sistema foi desenvolvido para o Instituto de Ciências Matemáticas e de Computação (ICMC) da Universidade de São Paulo (USP).

13.1 Tecnologias Utilizadas

- Python 3.x
- Python-MIP (Mixed Integer Programming)
- Pandas (análise de dados)
- OpenPyXL (manipulação de Excel)
- CBC Solver (otimização)